第 14 章練習解答　保護

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS14-E01　Domain 與最小權限

1. 靜態D={readS,writeT}使A可寫T、B可讀S、C可做兩者。可在A/B分別給DA={readS}、DB={writeT}，C給DC=∅，以受控轉移提供必要資料；不把原始任意記憶體或全權handle穿過邊界。
2. 否。CPU mode只區分特權，process/domain仍需各自映射/權限；同process不同函式更不因user mode就有記憶體隔離。

檢查。 動態轉移機制本身也要授權。

OS14-E02　矩陣變更的精確結果

1. 第一步D1={r*,owner},D2={w},D3={r}。第二步以本題r*連同read能力整體移轉的明確模型：D1={owner},D2={w,r*},D3={r}。若實作把r與copy flag分開，須先另定transfer語義，不能默默保留來源r。
2. owner(F)可刪F column中D2的w；control(D2)可刪D2 row中的w。control只允許row刪除，不能用它新增x；若D1以owner(F)新增x則是另一條授權規則。

檢查。 同column copy不會創造對G的權限。

OS14-E03　Switch 可達不等於同時持有

1. 此刻A[D1,P]=∅所以直接print拒絕；若允許兩次switch，先到D2再到D3可print。權限分析需含domain transition的可達性。
2. 從D1可達集合起為{D1}，每次沿允許邊只新增D2；D2無通D3邊，歸納所有到達domain都在{D1,D2}。兩者P權皆空，故不能print。若仍允許owner重新加邊，則此證明前提失效。

檢查。 列出凍結矩陣與允許操作類型。

OS14-E04　ACL、capability 與既存 open

1. U1={F:r}；U2={F:rw,G:r}；U3={G:w}。每一非空矩陣entry恰出現一次；把row/column互換不會自行改授權。
2. write一直拒絕，因handle沒有w。read依撤銷契約：若只在open檢ACL且handle仍live，可能繼續；若每次使用驗generation且已撤該generation，則之後read拒絕。不能只由刪ACL推定所有既存handle即刻作廢。

檢查。 ACL語義不得套用到帶deny/順序規則的系統。

第 14 章練習解答　保護 （續）

OS14-E05　Generation 與 ABA 反例

1. 只驗5會讓舊cap取得G。重用時改table5=(G,g4,live)，3≠4故拒。必須在同一同步區驗版本並取得保持object存活的reference，否則檢查後仍可被換物件。
2. mod4從3→0→1→2→3，4次即重現。若舊cap仍可用就ABA。不能以有限counter證明無界安全；可在wrap前禁止重用該slot，或透過全域quiescence確認舊tokens已消失，再允許新世代。

檢查。 數字夠大降低機率/延後，不等於形式上永不wrap。

OS14-E06　RBAC 的 active roles

1. active roles沒有writeGrades故拒；從Backup換Grade必先停用Backup，檢查新active集合不含互斥pair，且Grade屬assigned(U)。assigned不等於當前active。
2. 本題只限制同session，兩session各一role不違反字面規則；若目標是同一人不能同時握兩權，必須訂user-level或靜態分工constraint。機制不能替政策補上未定義範圍。

檢查。 明確分開UA、PA與session。

OS14-E07　Java 歷史 stack inspection

1. 普通：N通過→L通過→A拒，結果deny。L privileged：N通過→L驗p通過且為邊界→stop/allow；A不再檢查。
2. L無p則在L先拒絕，privileged不會創造p。若無可停止的privileged邊界，所有frame通過後仍檢inherited context；它無p則拒絕。
3. 低權A可借L已有的較大網路scope觸達自己不應觸達的目標；stack規則仍正確執行但服務政策失效。L須限制實際addr/port與操作，不能把未驗證caller參數當作授權依據。

檢查。 privileged caller自身必須具p。

OS14-E08　授權傳播與資訊流

1. 缺copy/owner等授權規則時不能授F:r給D2；但可合法readF取得bytes，再writeG，因此資訊仍可傳播。copy right不是資料複製的控制。
2. attenuation歸納依賴每個新增r都有來源既存r；owner可在自己的column新增尚無domain持有的有效right，此步破壞前提。須另證owner被授予的造權範圍/政策，不能沿用原定理。

檢查。 權限安全與機密資訊流分開論證。

第 14 章練習解答　保護 （續）

原創驗算程式附錄: access_matrix.py

```
"""Access matrix and revocation checks.

Run:
    python access_matrix.py
"""

Matrix = dict[str, dict[str, set[str]]]


def allowed(matrix: Matrix, domain: str, obj: str, right: str) -> bool:
    return right in matrix.get(domain, {}).get(obj, set())


def revoke(matrix: Matrix, domain: str, obj: str, right: str) -> None:
    rights = matrix.get(domain, {}).get(obj, set())
    rights.discard(right)


def main() -> None:
    matrix: Matrix = {
        "D1": {"F1": {"r", "w"}, "F2": set()},
        "D2": {"F1": {"r"}, "F2": {"w"}},
    }
    assert allowed(matrix, "D1", "F1", "w")
    revoke(matrix, "D1", "F1", "w")
    assert not allowed(matrix, "D1", "F1", "w")
    assert allowed(matrix, "D1", "F1", "r")
    assert allowed(matrix, "D2", "F2", "w")
    print(matrix)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 14 章練習解答　保護 （續）

原創驗算程式附錄: protection_models.py

```
"""Finite teaching cases; authority safety proofs are in the chapter."""
def reachable(start,edges):
 seen={start};stack=[start]
 while stack:
  for target in edges.get(stack.pop(),()):
   if target not in seen:seen.add(target);stack.append(target)
 return seen
def resolve(cap,entry):
 slot,g,right=cap;obj,version,live,rights=entry
 return obj if live and g==version and right in rights else None
def inspect(frames,inherited):
 for permitted,privileged in frames:
  if not permitted:return False
  if privileged:return True
 return inherited
if __name__=='__main__':
 assert reachable('D1',{'D1':['D2'],'D2':['D3']})=={'D1','D2','D3'}
 assert reachable('D1',{'D1':['D2']})=={'D1','D2'}
 assert resolve((7,12,'r'),('F',12,True,{'r'}))=='F'
 assert resolve((7,12,'r'),('G',13,True,{'r'})) is None
 assert not inspect([(True,False),(True,False),(False,False)],True)
 assert inspect([(True,False),(True,True),(False,False)],True)
 assert not inspect([(True,False),(False,True),(False,False)],True)
 assert not inspect([(True,False)],False)
 print('switch reachability, versioned handle and four stack cases passed')
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study